ANGELHACK × IBM BOB HACKATHON 2026 · THEME 1: EXPLORE, FIX, AND BUILD

OpenCut × IBM Bob

We used IBM Bob to understand an unfamiliar open-source video editor, audit its timeline, prove a real bug with a failing test, and fix it.

Kornelia Buszka · Madaline Perry

TEAM AND PROJECT

Early-career builders, an unfamiliar codebase

The team

Kornelia Buszka
Madaline Perry

The project

OpenCut is a free, privacy-first, open-source CapCut alternative. We forked the stable “classic” version and used IBM Bob to explore it, find bugs, and fix one.

THE CHALLENGE

A real, complex app we had never seen

1,567
commits of history before we started

3
languages and layers: Next.js, TypeScript, Rust → WASM

30+
storage schema migrations, with strict house rules

Custom lint rules, time stored as integer “ticks”, and a mandatory Command pattern for undo — easy to break without understanding it first.

STEP 1 · EXPLORE

Bob turned the codebase into a map

`/init`

Generated AGENTS.md plus per-mode rules for Ask, Plan and Agent modes, so Bob kept project context in every chat.

`Ask mode`

Traced the full 7-step flow from dropping a video file to it appearing on the timeline, with file paths.

`docs/architecture.md`

A Mermaid diagram of 7 layers and 5 data flows, now a guide for future contributors.

STEP 2 · AUDIT

Bob found 8 timeline issues, ranked by effort

#	Effort	Where	Issue
1	Easy	move-elements.ts	Command throws instead of returning, skipping undo
2	Easy	scenes-manager.ts	Hooks can crash when no scene is active
3	Easy	timeline-manager.ts	Silent path drops selection updates
4	Medium	move-elements.ts	Moves don’t validate overlap themselves
5	Medium	apply.ts	Audio track index formula is fragile
6	Medium	main-track.ts	Gap can remain at timeline start after delete
7	Hard	split-elements.ts	Split can create an invisible zero-length clip
8	Chosen	resolve.ts	Explicit placement skips the overlap check

THE BUG · #8

Dropping a clip could silently stack it on another

The timeline places clips with several strategies. Three of them call canPlaceTimeSpansOnTrack to prevent overlaps.

The explicit strategy — used when you drag onto a track or double-click an asset — skipped it.

User impact

Two clips could occupy the same moment on an overlay track. Both render on top of each other, with no warning, and the edit looks broken.

STEP 3 · PROVE, THEN FIX

A failing test before any code change

1 · Reproduce

explicit-overlap.test.ts: overlap case **fails**, control case passes. Bug confirmed.

2 · Plan, then fix

Plan mode designed the smallest fix; Agent mode added the overlap check to the explicit branch.

3 · Verify

4-line fix applied. Test now passes: **2 pass, 0 fail** in 250 ms.

Bob also solved a test-infrastructure problem: it mocked the WASM module so tests run without loading the binary.

HOW WE USED BOB

Bob was the workflow, not an add-on

Bob feature	What it did for us
/init + mode rules	Persistent project context and house rules in every chat
Ask mode + @mentions	Explained flows, scoped to folders to save Bobcoins
Agent mode	Ranked bug audit, wrote and ran the reproducing test
Plan mode	Designed the smallest safe fix before any code
/review + commits	Reviewed changes and wrote commit messages
Docs generation	Architecture diagram, README section, BOB_LOG.md

TECH STACK

What we built with

AI tooling

IBM Bob IDE (Pro+) — Ask, Plan and Agent modes, /init, /review

App

Next.js, React, TypeScript, Rust compiled to WASM, mediabunny

Testing and quality

Bun test runner, ESLint with custom rules, Prettier and Biome

Infrastructure

Bun, Turborepo, Docker Compose, IndexedDB and OPFS, GitHub

KEY LEARNINGS

What three weeks with Bob taught us

01 **Verify, don’t trust.** Bob’s audit was a list of hypotheses; a failing test turned one into a fact.

02 **Context beats long prompts.** /init rules and @mentions made answers precise and saved Bobcoins.

03 **Plan before Agent.** Small, scoped tasks finished fast; broad ones stalled and burned credits.

FUTURE ROADMAP

Seven more bugs, same playbook

Next

Fix #6 (gap at timeline start) and #7 (zero-length split), each with a failing test first.

Then

Harden commands #1 and #4, and add a user-facing message when a drop is rejected.

Later

Package our audit as a reusable Bob skill, so any team can run “find, prove, fix” on their timeline code.

THANK YOU

Explore. Prove. Fix.